Flutter Community

Flutter + Firebase + Mobile Development + Mobile App Development + Dart +

Using Flutter with Firebase: A Developer's Guide



Gulshan Yadav Follow 20 min read · Mar 19, 2026

30

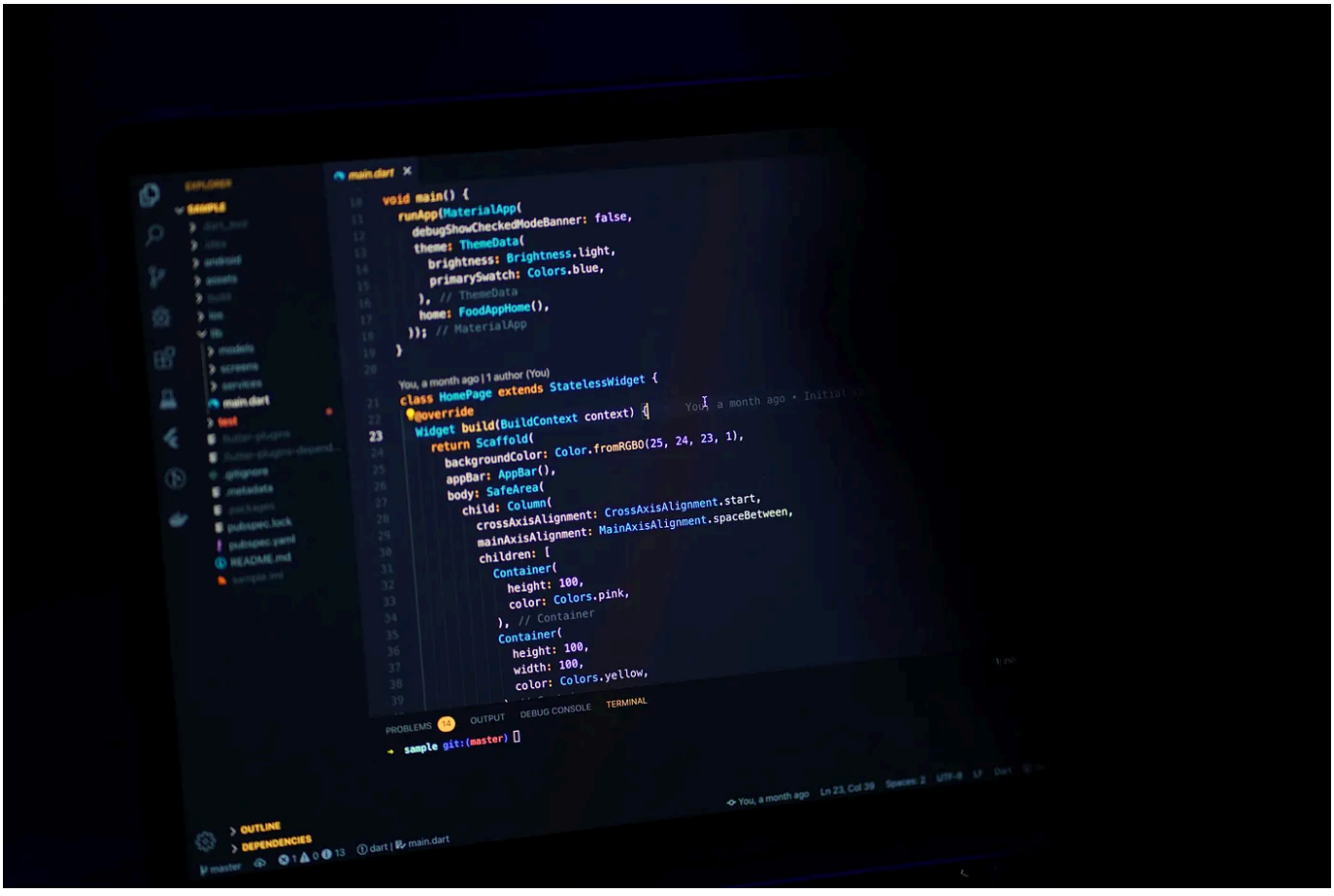


Photo by Mohammad Rahmani on Unsplash

Three years ago, I was building a food delivery app for a client in Bangalore. The requirements were standard: user authentication, real-time order tracking, image uploads for restaurant menus, push notifications, and analytics. The client wanted it done in six weeks. I had two choices: build a custom backend from scratch (Node.js, PostgreSQL, Redis, S3, a notification service, an auth system) or use Firebase. I chose Firebase, and I shipped the entire app in four weeks. That decision changed how I approach mobile development.

Since then, I've used Firebase in over 15 Flutter projects. Some were small MVPs with a couple hundred users, others were production apps serving 50,000+ daily active users. I've hit Firebase's limits, found workarounds, gotten burned by unexpected billing spikes, and learned the hard way which security rules actually protect your data. This guide is everything I wish someone had told me before I wrote my first `firebase init`.

Firebase isn't perfect. No backend is. But for Flutter developers who want to move fast, ship features, and not worry about server infrastructure, Firebase gives you an unfair advantage. Let me show you how to use it properly.

Why Firebase + Flutter Works So Well

The Problem: Building a backend from scratch for every mobile app takes weeks. You need auth, a database, file storage, push notifications, analytics, and crash reporting. That's six services to set up, maintain, and scale.

The Solution: Firebase gives you all six out of the box, with official Flutter packages maintained by Google, the same company that builds Flutter.

```
Firebase Services Overview:
+-----------------------------------------------------+
|                   Firebase                          |
|                                                     |
|  +--------------+  +-------------+  +------------+ |
|  |    Auth      |  |  Firestore  |  |  Storage   | |
|  | Email/Pass   |  | NoSQL DB    |  | Files/Imgs | |
|  | Google       |  | Real-time   |  | Up to 5TB  | |
|  | Phone        |  | Offline     |  | CDN backed | |
|  +--------------+  +-------------+  +------------+ |
|                                                     |
|  +--------------+  +-------------+  +------------+ |
|  | Cloud Funcs  |  |    FCM      |  | Analytics  | |
|  | Serverless   |  | Push Notifs |  | Events     | |
|  | Node.js/Py   |  | Topics      |  | Funnels    | |
|  | Triggers     |  | Scheduling  |  | Audiences  | |
|  +--------------+  +-------------+  +------------+ |
|                                                     |
|  +--------------+  +--------------+                 |
|  | Crashlytics  |  | Remote Cfg   |                 |
|  | Error logs   |  | A/B Testing  |                 |
|  | Stack traces |  | Feature flags|                 |
|  +--------------+  +--------------+                 |
+-----------------------------------------------------+
```

Here's why this combination specifically works:

1. **Same parent company.** Google builds both Flutter and Firebase. The `flutterfire` packages aren't community wrappers -- they're first-party, well-maintained, and updated alongside Firebase SDK releases.
2. **Real-time by default.** Firestore streams map perfectly to Flutter's `StreamBuilder` widget. You write one line and your UI updates automatically when data changes. No polling, no WebSocket boilerplate.
3. **Offline support built in.** Firestore caches data locally. Your Flutter app works offline with zero extra code. When the user reconnects, Firestore syncs automatically.

4. **One CLI to rule them all.** `flutterfire configure` auto-generates platform config files for Android, iOS, web, and macOS. No more manually downloading `google-services.json` and `GoogleService-Info.plist`.

Setting Up Firebase in a Flutter Project

Let me walk through the setup step by step. This has gotten much simpler since FlutterFire CLI was introduced.

Step 1: Install the Firebase CLI and FlutterFire CLI

```
# Install Firebase CLI (requires Node.js)
npm install -g firebase-tools

# Log in to your Firebase account
firebase login

# Install FlutterFire CLI globally
dart pub global activate flutterfire_cli
```

Step 2: Create a Firebase project

Go to the Firebase Console (console.firebase.google.com) and create a new project. Or use the CLI:

```
firebase projects:create my-flutter-app
```

Step 3: Configure your Flutter project

This is the magic step. Run this from your Flutter project root:

```
flutterfire configure --project=my-flutter-app
```

This command does the following automatically:

- Registers your app on Firebase for each platform (Android, iOS, web)
- Downloads and places config files
- Generates a `firebase_options.dart` file with all platform configs

Step 4: Add Firebase dependencies

```
# pubspec.yaml
dependencies:
  flutter:
    sdk: flutter
  firebase_core: ^2.27.0
  firebase_auth: ^4.17.0
  cloud_firestore: ^4.15.0
  firebase_storage: ^11.6.0
  firebase_messaging: ^14.7.0
  firebase_analytics: ^10.8.0
  firebase_crashlytics: ^3.4.0
  cloud_functions: ^4.6.0
  google_sign_in: ^6.2.1
```

Step 5: Initialize Firebase in your app

```
// main.dart
import 'package:flutter/material.dart';
import 'package:firebase_core/firebase_core.dart';
import 'package:firebase_crashlytics/firebase_crashlytics.dart';
```

```dart
import 'firebase_options.dart';

void main() async {
  WidgetsFlutterBinding.ensureInitialized();

  // Initialize Firebase with platform-specific options
  await Firebase.initializeApp(
    options: DefaultFirebaseOptions.currentPlatform,
  );

  // Pass all uncaught errors to Crashlytics
  FlutterError.onError = FirebaseCrashlytics.instance.recordFlutterFatalError;

  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({Key? key}) : super(key: key);

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Firebase Flutter App',
      theme: ThemeData(
        colorSchemeSeed: Colors.blue,
        useMaterial3: true,
      ),
      home: const AuthGate(),
    );
  }
}
```

That’s it. Five steps and you have Firebase running in your Flutter app across all platforms. When I started using Firebase with Flutter in 2021, this setup took an hour of manual config file copying. Now it takes under five minutes.Firebase Authentication

Auth is usually the first thing you add. Firebase Auth supports email/password, Google Sign-In, Apple Sign-In, phone number, and more. Here’s how I implement it in production apps.

Email/Password Authentication

```dart
// auth_service.dart
import 'package:firebase_auth/firebase_auth.dart';

class AuthService {
  final FirebaseAuth _auth = FirebaseAuth.instance;

  // Stream of auth state changes -- plug this into StreamBuilder
  Stream<User?> get authStateChanges => _auth.authStateChanges();

  // Get current user
  User? get currentUser => _auth.currentUser;

  // Register with email and password
  Future<UserCredential?> registerWithEmail({
    required String email,
    required String password,
    required String displayName,
  }) async {
    try {
      final credential = await _auth.createUserWithEmailAndPassword(
        email: email,
        password: password,
      );

      // Update display name after registration
      await credential.user?.updateDisplayName(displayName);

      // Send email verification
      await credential.user?.sendEmailVerification();

      return credential;
    } on FirebaseAuthException catch (e) {
      // Handle specific errors
      switch (e.code) {
        case 'weak-password':
          throw AuthException('Password must be at least 6 characters');
        case 'email-already-in-use':
          throw AuthException('An account with this email already exists');
        case 'invalid-email':
          throw AuthException('Please enter a valid email address');
        default:
          throw AuthException('Registration failed: ${e.message}');
      }
    }
```

```
  }

  // Sign in with email and password
  Future<UserCredential?> signInWithEmail({
    required String email,
    required String password,
  }) async {
    try {
      return await _auth.signInWithEmailAndPassword(
        email: email,
        password: password,
      );
    } on FirebaseAuthException catch (e) {
      switch (e.code) {
        case 'user-not-found':
          throw AuthException('No account found with this email');
        case 'wrong-password':
          throw AuthException('Incorrect password');
        case 'user-disabled':
          throw AuthException('This account has been disabled');
        case 'too-many-requests':
          throw AuthException('Too many attempts. Try again later');
        default:
          throw AuthException('Sign in failed: ${e.message}');
      }
    }
  }

  // Password reset
  Future<void> sendPasswordReset(String email) async {
    await _auth.sendPasswordResetEmail(email: email);
  }

  // Sign out
  Future<void> signOut() async {
    await _auth.signOut();
  }
}

// Custom exception for clean error handling in UI
class AuthException implements Exception {
  final String message;
  AuthException(this.message);

  @override
  String toString() => message;
}
```

Google Sign-In

Google Sign-In requires the `google_sign_in` package alongside Firebase Auth. Here's a complete implementation:

```
// google_auth_service.dart
import 'package:firebase_auth/firebase_auth.dart';
import 'package:google_sign_in/google_sign_in.dart';

class GoogleAuthService {
  final FirebaseAuth _auth = FirebaseAuth.instance;
  final GoogleSignIn _googleSignIn = GoogleSignIn(
    scopes: ['email', 'profile'],
  );

  Future<UserCredential?> signInWithGoogle() async {
    try {
      // Trigger the Google Sign-In flow
      final GoogleSignInAccount? googleUser = await _googleSignIn.signIn();

      // User cancelled the sign-in
      if (googleUser == null) return null;

      // Get auth details from the Google Sign-In
      final GoogleSignInAuthentication googleAuth =
          await googleUser.authentication;

      // Create a Firebase credential from Google tokens
      final credential = GoogleAuthProvider.credential(
        accessToken: googleAuth.accessToken,
        idToken: googleAuth.idToken,
      );

      // Sign in to Firebase with the Google credential
      final userCredential = await _auth.signInWithCredential(credential);

      // First time sign-in? Create user profile in Firestore
      if (userCredential.additionalUserInfo?.isNewUser ?? false) {
        await _createUserProfile(userCredential.user!);
      }

      return userCredential;
    } catch (e) {
      throw AuthException('Google sign-in failed: $e');
    }
```

```
  }

  Future<void> _createUserProfile(User user) async {
    await FirebaseFirestore.instance
        .collection('users')
        .doc(user.uid)
        .set({
      'uid': user.uid,
      'email': user.email,
      'displayName': user.displayName,
      'photoURL': user.photoURL,
      'createdAt': FieldValue.serverTimestamp(),
      'lastLogin': FieldValue.serverTimestamp(),
    });
  }

  Future<void> signOut() async {
    await _googleSignIn.signOut();
    await _auth.signOut();
  }
}
```

Auth Gate Widget

This is a pattern I use in every Firebase Flutter app. It listens to auth state and routes users accordingly:

```
// auth_gate.dart
import 'package:flutter/material.dart';
import 'package:firebase_auth/firebase_auth.dart';

class AuthGate extends StatelessWidget {
  const AuthGate({Key? key}) : super(key: key);

  @override
  Widget build(BuildContext context) {
    return StreamBuilder<User?>(
      stream: FirebaseAuth.instance.authStateChanges(),
      builder: (context, snapshot) {
        // Still loading
        if (snapshot.connectionState == ConnectionState.waiting) {
          return const Scaffold(
```

```
            body: Center(child: CircularProgressIndicator()),
          );
        }

        // User is signed in
        if (snapshot.hasData) {
          return const HomePage();
        }

// User is not signed in
        return const LoginPage();
      },
    );
  }
}
```

Cloud Firestore: Real-Time Database

Firestore is where Firebase really shines for Flutter developers. The real-time listeners work beautifully with Flutter's reactive widget system. Here's a complete CRUD implementation.

Data Model and Service

```
// models/task.dart
import 'package:cloud_firestore/cloud_firestore.dart';

class Task {
  final String id;
  final String title;
  final String description;
  final bool isCompleted;
  final DateTime createdAt;
  final String userId;
  final List<String> tags;

  Task({
    required this.id,
    required this.title,
    required this.description,
```

```
    required this.isCompleted,
    required this.createdAt,
    required this.userId,
    this.tags = const [],
  });

  // Convert Firestore document to Task object
  factory Task.fromFirestore(DocumentSnapshot doc) {
    final data = doc.data() as Map<String, dynamic>;
    return Task(
      id: doc.id,
      title: data['title'] ?? '',
      description: data['description'] ?? '',
      isCompleted: data['isCompleted'] ?? false,
      createdAt: (data['createdAt'] as Timestamp).toDate(),
      userId: data['userId'] ?? '',
      tags: List<String>.from(data['tags'] ?? []),
    );
  }

  // Convert Task to Firestore map
  Map<String, dynamic> toFirestore() {
    return {
      'title': title,
      'description': description,
      'isCompleted': isCompleted,
      'createdAt': Timestamp.fromDate(createdAt),
      'userId': userId,
      'tags': tags,
    };
  }
}

// services/firestore_service.dart
import 'package:cloud_firestore/cloud_firestore.dart';
import 'package:firebase_auth/firebase_auth.dart';

class FirestoreService {
  final FirebaseFirestore _db = FirebaseFirestore.instance;

  // Reference to user's tasks collection
  CollectionReference<Map<String, dynamic>> get _tasksRef =>
      _db.collection('users')
          .doc(FirebaseAuth.instance.currentUser!.uid)
          .collection('tasks');

  // CREATE -- add a new task
  Future<String> addTask({
    required String title,
    required String description,
```

```
    List<String> tags = const [],
  }) async {
    final docRef = await _tasksRef.add({
      'title': title,
      'description': description,
      'isCompleted': false,
      'createdAt': FieldValue.serverTimestamp(),
      'userId': FirebaseAuth.instance.currentUser!.uid,
      'tags': tags,
    });
    return docRef.id;
  }

  // READ -- stream all tasks (real-time updates)
  Stream<List<Task>> streamTasks({bool? completedFilter}) {
    Query<Map<String, dynamic>> query = _tasksRef.orderBy('createdAt', descendin

    if (completedFilter != null) {
      query = query.where('isCompleted', isEqualTo: completedFilter);
    }

    return query.snapshots().map((snapshot) {
      return snapshot.docs.map((doc) => Task.fromFirestore(doc)).toList();
    });
  }

  // READ -- get a single task
  Future<Task?> getTask(String taskId) async {
    final doc = await _tasksRef.doc(taskId).get();
    if (doc.exists) {
      return Task.fromFirestore(doc);
    }
    return null;
  }

  // UPDATE -- toggle task completion
  Future<void> toggleTask(String taskId, bool isCompleted) async {
    await _tasksRef.doc(taskId).update({
      'isCompleted': isCompleted,
      'completedAt': isCompleted ? FieldValue.serverTimestamp() : null,
    });
  }

  // UPDATE -- edit task details
  Future<void> updateTask(String taskId, {
    String? title,
    String? description,
    List<String>? tags,
  }) async {
    final updates = <String, dynamic>{};
```

```
  if (title != null) updates['title'] = title;
  if (description != null) updates['description'] = description;
  if (tags != null) updates['tags'] = tags;
  updates['updatedAt'] = FieldValue.serverTimestamp();

  await _tasksRef.doc(taskId).update(updates);
}

// DELETE -- remove a task
Future<void> deleteTask(String taskId) async {
  await _tasksRef.doc(taskId).delete();
}

// BATCH -- complete all tasks at once
Future<void> completeAllTasks() async {
  final batch = _db.batch();
  final snapshot = await _tasksRef
      .where('isCompleted', isEqualTo: false)
      .get();

  for (final doc in snapshot.docs) {
    batch.update(doc.reference, {
      'isCompleted': true,
      'completedAt': FieldValue.serverTimestamp(),
    });
  }

  await batch.commit();
}

// QUERY -- search tasks by tag
Stream<List<Task>> streamTasksByTag(String tag) {
  return _tasksRef
      .where('tags', arrayContains: tag)
      .orderBy('createdAt', descending: true)
      .snapshots()
      .map((snapshot) {
    return snapshot.docs.map((doc) => Task.fromFirestore(doc)).toList();
  });
}

// PAGINATION -- load tasks in pages of 20
Future<List<Task>> getTasksPaginated({
  DocumentSnapshot? lastDocument,
  int limit = 20,
}) async {
  Query<Map<String, dynamic>> query = _tasksRef
      .orderBy('createdAt', descending: true)
      .limit(limit);
```

```
      if (lastDocument != null) {
        query = query.startAfterDocument(lastDocument);
      }

      final snapshot = await query.get();
      return snapshot.docs.map((doc) => Task.fromFirestore(doc)).toList();
    }
  }
```

Real-Time UI with StreamBuilder

Here's how to wire the Firestore stream to your Flutter UI:

```
// screens/task_list_screen.dart
class TaskListScreen extends StatelessWidget {
  final FirestoreService _firestoreService = FirestoreService();

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('My Tasks')),
      body: StreamBuilder<List<Task>>(
        stream: _firestoreService.streamTasks(),
        builder: (context, snapshot) {
          if (snapshot.hasError) {
            return Center(child: Text('Error: ${snapshot.error}'));
          }

          if (!snapshot.hasData) {
            return const Center(child: CircularProgressIndicator());
          }

          final tasks = snapshot.data!;

          if (tasks.isEmpty) {
            return const Center(child: Text('No tasks yet. Add one!'));
          }

          return ListView.builder(
            itemCount: tasks.length,
            itemBuilder: (context, index) {
              final task = tasks[index];
              return Dismissible(
```

```
                    key: Key(task.id),
                    onDismissed: (_) => _firestoreService.deleteTask(task.id),
                    background: Container(color: Colors.red),
                    child: CheckboxListTile(
                      title: Text(
                        task.title,
                        style: TextStyle(
                          decoration: task.isCompleted
                              ? TextDecoration.lineThrough
                              : null,
                        ),
                      ),
                      subtitle: Text(task.description),
                      value: task.isCompleted,
                      onChanged: (value) {
                        _firestoreService.toggleTask(task.id, value ?? false);
                      },
                    ),
                  );
                },
              );
            },
          ),
          floatingActionButton: FloatingActionButton(
            onPressed: () => _showAddTaskDialog(context),
            child: const Icon(Icons.add),
          ),
        );
      }
    }
```

One thing I learned the hard way: always use subcollections for user-specific data (like `users/{uid}/tasks` instead of a top-level `tasks` collection with a `userId` field). Subcollections make security rules simpler and queries faster because you're never scanning data that belongs to other users.Firebase Storage: File Uploads

Every app eventually needs file uploads. Profile pictures, documents, receipts — Firebase Storage handles it all with built-in CDN.

```dart
// services/storage_service.dart
import 'dart:io';
import 'package:firebase_storage/firebase_storage.dart';
import 'package:firebase_auth/firebase_auth.dart';
import 'package:image_picker/image_picker.dart';
import 'package:path/path.dart' as path;

class StorageService {
  final FirebaseStorage _storage = FirebaseStorage.instance;
  final String _uid = FirebaseAuth.instance.currentUser!.uid;

  // Upload profile picture with progress tracking
  Future<String> uploadProfilePicture(File file) async {
    final ext = path.extension(file.path);
    final ref = _storage.ref('users/$_uid/profile$ext');

    // Set metadata for proper content type
    final metadata = SettableMetadata(
      contentType: 'image/${ext.replaceAll('.', '')}',
      customMetadata: {
        'uploadedBy': _uid,
        'uploadedAt': DateTime.now().toIso8601String(),
      },
    );

    // Upload with progress monitoring
    final uploadTask = ref.putFile(file, metadata);

    uploadTask.snapshotEvents.listen((event) {
      final progress = event.bytesTransferred / event.totalBytes;
      print('Upload progress: ${(progress * 100).toStringAsFixed(1)}%');
    });

    // Wait for upload to complete
    await uploadTask;

    // Return download URL
    return await ref.getDownloadURL();
  }

  // Upload any file (documents, images, etc.)
  Future<String> uploadFile({
    required File file,
    required String folder,
    String? customName,
  }) async {
    final fileName = customName ?? path.basename(file.path);
    final ref = _storage.ref('users/$_uid/$folder/$fileName');
```

```
    await ref.putFile(file);
    return await ref.getDownloadURL();
  }

  // Pick and upload image from gallery
  Future<String?> pickAndUploadImage() async {
    final picker = ImagePicker();
    final pickedFile = await picker.pickImage(
      source: ImageSource.gallery,
      maxWidth: 1024,    // Resize to save storage costs
      maxHeight: 1024,
      imageQuality: 80,  // Compress to save bandwidth
    );

    if (pickedFile == null) return null;

    final file = File(pickedFile.path);
    return await uploadProfilePicture(file);
  }

  // Delete a file by its download URL
  Future<void> deleteFile(String downloadUrl) async {
    final ref = _storage.refFromURL(downloadUrl);
    await ref.delete();
  }

  // List all files in a folder
  Future<List<Reference>> listFiles(String folder) async {
    final result = await _storage.ref('users/$_uid/$folder').listAll();
    return result.items;
  }
}
```

Pro tip on storage costs: Always compress images before uploading. A 4MB photo from a phone camera becomes 200KB at 80% quality and 1024px max dimension. That's a 20x reduction in storage costs. I've seen apps where this single optimization cut the monthly Firebase bill by 60%.Cloud Functions: Serverless Backend Logic

Get Gulshan Yadav's stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email

Subscribe

Remember me for faster sign in

Cloud Functions let you run backend code without managing servers. I use them for things that shouldn't happen on the client: sending welcome emails, processing payments, aggregating data, and cleaning up orphaned documents.

```
// functions/index.js
const functions = require('firebase-functions');
const admin = require('firebase-admin');
admin.initializeApp();

// Trigger: when a new user is created
exports.onUserCreated = functions.auth.user().onCreate(async (user) => {
  // Create a user profile document in Firestore
  await admin.firestore().collection('users').doc(user.uid).set({
    email: user.email,
    displayName: user.displayName || '',
    photoURL: user.photoURL || '',
    createdAt: admin.firestore.FieldValue.serverTimestamp(),
    plan: 'free',
    taskCount: 0,
  });

  // Send welcome email (using your email service)
  // await sendWelcomeEmail(user.email, user.displayName);
  console.log(`User profile created for ${user.uid}`);
});

// Trigger: when a task is created, increment the counter
exports.onTaskCreated = functions.firestore
  .document('users/{userId}/tasks/{taskId}')
  .onCreate(async (snap, context) => {
    const userId = context.params.userId;
```

```
    await admin.firestore().collection('users').doc(userId).update({
      taskCount: admin.firestore.FieldValue.increment(1),
    });
  });

// Trigger: when a task is deleted, decrement the counter
exports.onTaskDeleted = functions.firestore
  .document('users/{userId}/tasks/{taskId}')
  .onDelete(async (snap, context) => {
    const userId = context.params.userId;

    await admin.firestore().collection('users').doc(userId).update({
      taskCount: admin.firestore.FieldValue.increment(-1),
    });
  });

// Callable function: generate a weekly report
exports.generateReport = functions.https.onCall(async (data, context) => {
  // Verify user is authenticated
  if (!context.auth) {
    throw new functions.https.HttpsError(
      'unauthenticated',
      'You must be signed in to generate a report.'
    );
  }

  const userId = context.auth.uid;
  const tasksRef = admin.firestore()
    .collection('users').doc(userId)
    .collection('tasks');

  const sevenDaysAgo = new Date();
  sevenDaysAgo.setDate(sevenDaysAgo.getDate() - 7);

  const completedTasks = await tasksRef
    .where('isCompleted', '==', true)
    .where('completedAt', '>=', sevenDaysAgo)
    .get();

  const totalTasks = await tasksRef.get();

  return {
    totalTasks: totalTasks.size,
    completedThisWeek: completedTasks.size,
    completionRate: totalTasks.size > 0
      ? ((completedTasks.size / totalTasks.size) * 100).toFixed(1)
      : '0',
    generatedAt: new Date().toISOString(),
  };
});
```

```
// Scheduled function: clean up old completed tasks every Sunday
exports.weeklyCleanup = functions.pubsub
  .schedule('0 2 * * 0')  // 2 AM every Sunday
  .timeZone('Asia/Kolkata')
  .onRun(async (context) => {
    const thirtyDaysAgo = new Date();
    thirtyDaysAgo.setDate(thirtyDaysAgo.getDate() - 30);

    const usersSnapshot = await admin.firestore().collection('users').get();

    for (const userDoc of usersSnapshot.docs) {
      const oldTasks = await userDoc.ref
        .collection('tasks')
        .where('isCompleted', '==', true)
        .where('completedAt', '<=', thirtyDaysAgo)
        .get();

      const batch = admin.firestore().batch();
      oldTasks.docs.forEach((doc) => batch.delete(doc.ref));
      await batch.commit();

      console.log(`Cleaned ${oldTasks.size} old tasks for user ${userDoc.id}`);
    }
  });
```

Calling a Cloud Function from Flutter:

```
// Calling the report generator from Flutter
import 'package:cloud_functions/cloud_functions.dart';

Future<Map<String, dynamic>> getWeeklyReport() async {
  final callable = FirebaseFunctions.instance.httpsCallable('generateReport');
  final result = await callable.call();
  return Map<String, dynamic>.from(result.data);
}
```

Push Notifications with FCM

Open in app ↗ Sign up Sign in

Medium Search Write

both Android and iOS. The setup has a few platform-specific steps, but the Flutter code is straightforward.

```
// services/notification_service.dart
import 'package:firebase_messaging/firebase_messaging.dart';
import 'package:flutter_local_notifications/flutter_local_notifications.dart';

class NotificationService {
  final FirebaseMessaging _messaging = FirebaseMessaging.instance;
  final FlutterLocalNotificationsPlugin _localNotifications =
      FlutterLocalNotificationsPlugin();

  Future<void> initialize() async {
    // Request permission (required on iOS, good practice on Android 13+)
    final settings = await _messaging.requestPermission(
      alert: true,
      badge: true,
      sound: true,
      provisional: false,
    );

    if (settings.authorizationStatus == AuthorizationStatus.authorized) {
      print('User granted notification permission');
    }

    // Get the FCM token for this device
    final token = await _messaging.getToken();
    print('FCM Token: $token');
    // Save this token to Firestore for your user
    await _saveTokenToFirestore(token);

    // Listen for token refresh (happens periodically)
    _messaging.onTokenRefresh.listen(_saveTokenToFirestore);

    // Handle foreground messages
    FirebaseMessaging.onMessage.listen(_handleForegroundMessage);

    // Handle background/terminated message taps
    FirebaseMessaging.onMessageOpenedApp.listen(_handleMessageTap);

    // Check if app was opened from a terminated state via notification
    final initialMessage = await _messaging.getInitialMessage();
    if (initialMessage != null) {
      _handleMessageTap(initialMessage);
```

```
    }

    // Initialize local notifications for foreground display
    await _initLocalNotifications();
  }

  Future<void> _initLocalNotifications() async {
    const androidSettings = AndroidInitializationSettings('@mipmap/ic_launcher');
    const iosSettings = DarwinInitializationSettings();

    await _localNotifications.initialize(
      const InitializationSettings(
        android: androidSettings,
        iOS: iosSettings,
      ),
    );
  }

  void _handleForegroundMessage(RemoteMessage message) {
    // Show a local notification since FCM doesn't display in foreground
    if (message.notification != null) {
      _localNotifications.show(
        message.hashCode,
        message.notification!.title,
        message.notification!.body,
        const NotificationDetails(
          android: AndroidNotificationDetails(
            'default_channel',
            'Default',
            importance: Importance.high,
            priority: Priority.high,
          ),
        ),
      );
    }
  }

  void _handleMessageTap(RemoteMessage message) {
    // Navigate to the relevant screen based on notification data
    final data = message.data;
    if (data.containsKey('taskId')) {
      // Navigate to task detail screen
      // Use your router/navigator here
    }
  }

  Future<void> _saveTokenToFirestore(String? token) async {
    if (token == null) return;
    final uid = FirebaseAuth.instance.currentUser?.uid;
    if (uid == null) return;
```

```
      await FirebaseFirestore.instance.collection('users').doc(uid).update({
        'fcmTokens': FieldValue.arrayUnion([token]),
      });
    }

    // Subscribe to a topic (e.g., "news", "offers")
    Future<void> subscribeToTopic(String topic) async {
      await _messaging.subscribeToTopic(topic);
    }

    // Unsubscribe from a topic
    Future<void> unsubscribeFromTopic(String topic) async {
      await _messaging.unsubscribeFromTopic(topic);
    }
}
```

Don't forget the background message handler. This must be a top-level function (not inside a class):

```
// main.dart -- add this outside of any class
@pragma('vm:entry-point')
Future<void> _firebaseMessagingBackgroundHandler(RemoteMessage message) async {
  await Firebase.initializeApp(options: DefaultFirebaseOptions.currentPlatform);
  print('Handling background message: ${message.messageId}');
}

void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp(options: DefaultFirebaseOptions.currentPlatform);

  // Register the background handler
  FirebaseMessaging.onBackgroundMessage(_firebaseMessagingBackgroundHandler);

  runApp(const MyApp());
}
```

Firebase Analytics and Crashlytics

Analytics and Crashlytics are free and take five minutes to set up. There's no reason not to include them in every app.

Analytics

```dart
// services/analytics_service.dart
import 'package:firebase_analytics/firebase_analytics.dart';

class AnalyticsService {
  final FirebaseAnalytics _analytics = FirebaseAnalytics.instance;

  // Track screen views
  Future<void> logScreenView(String screenName) async {
    await _analytics.logScreenView(screenName: screenName);
  }

  // Track custom events
  Future<void> logTaskCreated(String taskType) async {
    await _analytics.logEvent(
      name: 'task_created',
      parameters: {'task_type': taskType},
    );
  }

  Future<void> logTaskCompleted(int daysToComplete) async {
    await _analytics.logEvent(
      name: 'task_completed',
      parameters: {'days_to_complete': daysToComplete},
    );
  }

  // Track user properties for segmentation
  Future<void> setUserPlan(String plan) async {
    await _analytics.setUserProperty(name: 'plan', value: plan);
  }
}
```

Crashlytics

Crashlytics was already initialized in our main.dart above. Here's how to add custom crash context:

```
// Add user context to crash reports
FirebaseCrashlytics.instance.setUserIdentifier(user.uid);
FirebaseCrashlytics.instance.setCustomKey('plan', 'premium');

// Catch and report non-fatal errors
try {
  await riskyOperation();
} catch (e, stackTrace) {
  await FirebaseCrashlytics.instance.recordError(
    e,
    stackTrace,
    reason: 'Failed during riskyOperation',
  );
}
```

Security Rules: The Most Ignored Part

I've audited Firebase projects where the security rules were literally allow read, write: if true;. That's like leaving your house with the front door wide open and a sign that says "free stuff inside." Here are the rules I use in production:

```
// firestore.rules
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {

    // Users can only read/write their own profile
    match /users/{userId} {
      allow read: if request.auth != null && request.auth.uid == userId;
      allow create: if request.auth != null && request.auth.uid == userId;
      allow update: if request.auth != null && request.auth.uid == userId
                    && !request.resource.data.diff(resource.data).affectedKeys()
                        .hasAny(['createdAt', 'uid']); // Can't change immutable

      // Tasks subcollection -- only the owner
      match /tasks/{taskId} {
        allow read, write: if request.auth != null
                           && request.auth.uid == userId;
```

```
        // Validate task data on create
        allow create: if request.auth != null
                      && request.auth.uid == userId
                      && request.resource.data.keys().hasAll(['title', 'isComple
                      && request.resource.data.title is string
                      && request.resource.data.title.size() > 0
                      && request.resource.data.title.size() <= 200;
      }
    }

    // Shared data (read-only for authenticated users)
    match /categories/{categoryId} {
      allow read: if request.auth != null;
      allow write: if false; // Only Cloud Functions can write
    }

    // Deny everything else by default
    match /{document=**} {
      allow read, write: if false;
    }
  }
}
```

```
// storage.rulesrules_version = '2';
service firebase.storage {
  match /b/{bucket}/o {
    // User files -- only the owner can read/write
    match /users/{userId}/{allPaths=**} {
      allow read: if request.auth != null && request.auth.uid == userId;
      allow write: if request.auth != null && request.auth.uid == userId
                   && request.resource.size < 10 * 1024 * 1024  // Max 10MB
                   && request.resource.contentType.matches('image/.*'); // Image
    }

    // Deny everything else
    match /{allPaths=**} {
      allow read, write: if false;
    }
  }
}
```

Three rules I follow for security:

Default deny. Start with allow read, write: if false; for the catch-all and only open specific paths.

Validate data shape. Check that required fields exist and have the right types. Don’t trust the client.

Never allow writes to admin fields from the client. Fields like plan, role, and createdAt should only be writable by Cloud Functions.

Firebase vs Supabase vs Custom Backend

I get asked this a lot: “Should I use Firebase or Supabase?” Here’s my honest comparison after using both in production:

Feature	Firebase	Supabase	Custom Backend
Database	NoSQL (Firestore)	PostgreSQL (relational)	Your choice
Real-time	Built-in (excellent)	Built-in (good)	WebSockets/SSE (manual setup)
Auth	Excellent (many providers + phone auth)	Good (growing providers)	Build it yourself
File Storage	Built-in CDN	Built-in	S3 + CloudFront
Serverless	Cloud Functions (mature)	Edge Functions (newer)	AWS Lambda / your server
Flutter Support	First-party (Google-built)	Community (supabase_fl)	REST/GraphQL (manual)
Pricing	Pay-per-use (can spike)	Generous free tier, then $25	Server costs (predictable)
Vendor Lock-in	High	Low (Postgres	None

```
|                  |                | is portable)   |                  |
+------------------+----------------+----------------+------------------+
| Self-hosting     | No             | Yes            | Yes              |
+------------------+----------------+----------------+------------------+
| Learning Curve   | Low            | Medium         | High             |
+------------------+----------------+----------------+------------------+
```

My recommendation:

Use Firebase when: you need to move fast, your data model is hierarchical (users -> orders -> items), you want the best Flutter integration, or your team is small and doesn't want to manage infrastructure.

Use Supabase when: you need relational data (joins, foreign keys, complex queries), you want to self-host, you're worried about vendor lock-in, or you have PostgreSQL experience.

Use a custom backend when: you have very specific requirements (e.g., processing video, ML inference), you need full control over your data pipeline, or your scale requires custom optimization.

For most Flutter apps, especially MVPs and early-stage products, Firebase is still the fastest path to production.

Cost Optimization Tips

Firebase pricing caught me off guard on my third project. The app had 10,000 users, and one month the Firestore bill jumped from $15 to $180 because of a bug that caused infinite document reads in a loop. Here's what I've learned about keeping costs down:

1. Use Firestore offline persistence wisely. Firestore caches data locally. If your users open the app and the data hasn't changed, the reads come from cache (free). But if you call .get() with GetOptions(source: Source.server) every time, you're paying for unnecessary reads.

2. Paginate everything. Never load an entire collection. Always use .limit() and implement pagination. Loading 1,000 documents when the user only sees 20 is throwing money away.

3. Use subcollections instead of arrays. If you store a list of 100 items as an array field, every time you read the parent document, you read all 100 items. With a subcollection, you only read the items you need.

4. Composite indexes over multiple queries. Instead of running three queries and merging results on the client (3x the reads), create a composite index and run one query.

5. Use Cloud Functions for aggregation. Don't calculate totals on the client by reading all documents. Use Cloud Functions triggers to maintain running counters (like the taskCount example above).

6. Monitor your usage dashboard. Check the Firebase Console -> Usage and billing weekly. Set up budget alerts at 50% and 80% of your monthly budget. I set mine at $10 and $20 so I'm never surprised.

7. Use the Blaze plan from day one. The Spark (free) plan has hard limits. The Blaze (pay-as-you-go) plan still includes the same free tier (50K reads/day, 20K writes/day, 1GB storage) but doesn't cut you off when you exceed it. You just pay for the overage.

```
Firebase Free Tier (Daily Limits):
+-----------------------------+---------+
| Service                     | Free    |
+-----------------------------+---------+
| Firestore Reads             | 50,000  |
| Firestore Writes            | 20,000  |
| Firestore Deletes           | 20,000  |
| Firestore Storage           | 1 GB    |
| Cloud Storage               | 5 GB    |
| Cloud Storage Downloads     | 1 GB/day|
| Authentication              |Unlimited|
| Cloud Functions Invocations |2M/month |
| FCM Messages                |Unlimited|
| Analytics                   |Unlimited|
| Crashlytics                 |Unlimited|
+-----------------------------+---------+
```

For a typical app with 10,000 monthly active users, you're looking at roughly $5–25/month on the Blaze plan. That's less than a single Starbucks run per week for a full backend.

Common Pitfalls and Debugging

After 15 Flutter + Firebase projects, here are the mistakes I see developers make repeatedly:

1. Not handling the cold start. When your app launches, FirebaseAuth.instance.currentUser might be null for a split second, even if the user is logged in. Always use authStateChanges() stream instead of checking currentUser directly.

2. Forgetting composite indexes. If your Firestore query uses where() + orderBy() on different fields, you need a composite index. Firestore will throw an error with a direct link to create the index — follow that link, don't try to guess the index configuration.

3. Not testing security rules. Firebase has a local emulator suite. Use it. Test your security rules before deploying.

```
# Start the Firebase emulator suite
firebase emulators:start

# Run your app against the emulator
# In your Flutter app:

// Point Firebase to local emulators during development
if (kDebugMode) {
  await FirebaseAuth.instance.useAuthEmulator('localhost', 9099);
  FirebaseFirestore.instance.useFirestoreEmulator('localhost', 8080);
  await FirebaseStorage.instance.useStorageEmulator('localhost', 9199);
  FirebaseFunctions.instance.useFunctionsEmulator('localhost', 5001);
}
```

4. Storing sensitive data in Firestore without encryption. Firestore is encrypted at rest, but anyone with read access to a document sees all fields. Don’t store passwords, API keys, or unencrypted payment info in Firestore. Use Cloud Functions to handle sensitive operations server-side.

5. Not using batch writes for multiple operations. If you need to update 50 documents, don’t write 50 individual update calls. Use WriteBatch — it’s atomic (all succeed or all fail) and counts as a single write operation for billing.

6. Ignoring the 1MB document size limit. Firestore documents have a 1MB size limit. If you’re storing large blobs of text, arrays with thousands of items, or base64-encoded images in a document, you’ll hit this wall. Use Storage for files and subcollections for large lists.

7. Not cleaning up listeners. Every snapshots() stream is an active connection. If you navigate away from a screen but don't cancel the stream subscription, you're paying for reads in the background. Use StreamSubscription and cancel in dispose(), or let StreamBuilder handle it automatically (it cancels when the widget is removed from the tree).

Putting It All Together

Here's the architecture I use for production Flutter + Firebase apps:

```
Project Structure:
lib/
  main.dart                    # App entry point, Firebase init
  firebase_options.dart        # Auto-generated by FlutterFire CLI
  app.dart                     # MaterialApp, routing, theming

  models/                      # Data classes
    user.dart
    task.dart

  services/                    # Firebase service wrappers
    auth_service.dart
    firestore_service.dart
    storage_service.dart
    notification_service.dart
    analytics_service.dart

  providers/                   # State management (Riverpod/Provider)
    auth_provider.dart
    task_provider.dart

  screens/                     # UI screens
    auth/
      login_screen.dart
      register_screen.dart
    home/
      home_screen.dart
      task_list_screen.dart
      task_detail_screen.dart
    profile/
      profile_screen.dart
```

```
  widgets/                      # Reusable UI components
    task_card.dart
    loading_overlay.dart

functions/                      # Cloud Functions (Node.js)
  index.js
  package.json

firestore.rules                 # Firestore security rules
storage.rules                   # Storage security rules
firebase.json                   # Firebase project config
```

The key principle: services wrap Firebase, providers wrap services, screens consume providers. This way, if you ever switch from Firebase to Supabase or a custom backend, you only change the service layer. Everything else stays the same.

Conclusion

Firebase and Flutter together give you the ability to build full-featured, production-ready mobile apps without setting up or maintaining a single server. After seven years of building mobile apps and three years of using this specific combination, I can tell you that the developer experience keeps getting better with each release.

Key takeaways from this guide:

FlutterFire CLI (flutterfire configure) eliminates all manual platform configuration — use it

Firebase Auth with authStateChanges() + StreamBuilder gives you reactive auth UI with minimal code

Firestore real-time streams map perfectly to Flutter's widget model — use StreamBuilder everywhere

Subcollections over arrays for user-specific data keeps queries fast and security rules simple

Cloud Functions handle server-side logic without managing infrastructure

FCM with flutter_local_notifications covers both foreground and background push notifications

Security rules should default to deny everything and only open specific paths with validation

Use the Firebase emulator suite for local development and testing security rules

Monitor your usage dashboard weekly and set up billing alerts to avoid surprises

Always compress images before uploading to Storage — it can cut your bill by over half

The combination isn't going anywhere. Google continues to invest heavily in both Flutter and Firebase, and the integration keeps getting tighter. If you're building a mobile app in 2026 and want to ship fast with a reliable backend, this is the stack to bet on.

If this guide saved you time, give it some claps and share it with your team. Follow me for more hands-on Flutter guides, backend architecture deep

dives, and practical mobile development tips. I publish new content every week based on real production experience, not just tutorials copied from docs.

Flutter + Firebase + Mobile Development + Mobile App Development + Dart +



Written by Gulshan Yadav

243 followers · 48 following

Follow

Founder @ Misar.io, Assisters.io. AI systems builder · 7 years in production. RAG, self-hosted infra, architectures. 📪 Deep-dives → mrgulshanyadav.substack.com



Published in Flutter Community

71K followers · Last published Sep 20, 2026

Follow

Articles and Stories from the Flutter Community